

# Zaawansowana automatyzacja

Playwright + TypeScript

**AAA**   **Page Object**   **SMURF**

# Trzy części warsztatu

Każdy temat kończy się zadaniem w projekcie.

**01 AAA** Przygotowanie, zachowanie i wynik testu. Zadanie 01.

**02 Page Object** Locatory i interakcje z formularzem. Zadanie 02.

**03 SMURF** Jakość testów, fixtures i API. Zadania 03–10.

Efekt końcowy: porównanie tego samego logowania w trzech wariantach.

# Mapa aktualnego projektu

| Miejsce                              | Rola w warsztacie                      |
|--------------------------------------|----------------------------------------|
| tests/patterns/                      | Gotowe przykłady AAA i beforeEach      |
| src/pageobjects/WorkshopLoginPage.ts | Szkielet Page Object do uzupełnienia   |
| fixtures/smurf/fixtures.ts           | Szkielet testUser i loginPage          |
| app/ i utils/generators/             | Aplikacja logowania oraz DataGenerator |
| tests/workshop/                      | Nowe testy pisane przez uczestnika     |

# Formularz przykładowy i lokalna aplikacja

Wybór ścieżki określa selektor i oczekiwany komunikat.

|                | Przykłady w tests/patterns | Aplikacja w app/               |
|----------------|----------------------------|--------------------------------|
| Źródło strony  | HTML przez page.route      | Serwer npm run app             |
| Selektor błędu | flash-message              | login-error                    |
| Komunikat      | Invalid credentials        | Nieprawidłowy login lub hasło. |
| Zakres         | Organizacja kodu testu     | Rzeczywiste logowanie i API    |

Ćwiczenia na lokalnej aplikacji korzystają z serwera i jego rzeczywistych odpowiedzi.

# Przygotowanie środowiska

Polecenia z katalogu głównego projektu, przykład dla PowerShell.

```
npm ci
Copy-Item .env.sample .env
npm run install:browsers

# Osobny terminal dla ćwiczeń z aplikacją
npm run app
```

UI: <http://localhost:3000/login>

API: [/api/users](#), [/api/users/<email>](#)

# Uruchamianie wybranego testu

Po każdej małej zmianie uruchamiamy konkretny plik.

```
npx playwright test tests/patterns/login-aaa.spec.ts --project=chromium

# Po napisaniu własnego pliku
npx playwright test tests/workshop/login.spec.ts --project=chromium

npm run test:report
```

W tej wersji login-page-object.spec.ts importuje brakujący LoginPage.  
Pełne npm test wymaga dostosowania tego przykładu.

# AAA

## Czytelny podział scenariusza testowego

Arrange, Act, Assert

Przykład lokalnego logowania i zadanie 01

# Arrange, Act, Assert

Granice faz pomagają zrozumieć, co test sprawdza.

## Arrange

Dane, stan konta i otwarcie właściwego ekranu.

## Act

Testowane zachowanie, np. wysłanie formularza logowania.

## Assert

Obserwowalny efekt: komunikat błędu albo Dashboard.

Kilka kroków formularza może składać się na jedno zachowanie użytkownika.

# AAA w lokalnym teście logowania

Przykład: istniejący użytkownik podaje błędne hasło.

```
test('błąd dla nieprawidłowego hasła', async ({ page }) => {  
  // Arrange  
  await page.goto('/login');  
  
  // Act  
  await page.getByLabel('Email').fill('workshop.user@example.com');  
  await page.getByLabel('Password').fill('wrong-password');  
  await page.getByRole('button', { name: 'Sign in' }).click();  
  
  // Assert  
  await expect(page.getByTestId('login-error'))  
    .toHaveText('Nieprawidłowy login lub hasło.');
```

# Test logowania w układzie AAA

## Punkt startowy

Wzór: `tests/patterns/login-aaa.spec.ts`

Nowy plik: `tests/workshop/login.spec.ts`

## Do wykonania

1. Napisz test nieznanego użytkownika dla lokalnej aplikacji.
2. Oddziel dane i nawigację od wysłania formularza.
3. Sprawdź login-error i dokładny polski komunikat.

## Kryteria ukończenia

Nazwa opisuje zachowanie.

Act obejmuje jedno logowanie.

Wybrany test przechodzi.

```
npx playwright test tests/workshop/login.spec.ts --project=chromium
```

# Page Object

Nazwane interakcje z ekranem

WorkshopLoginPage i granice odpowiedzialności  
Zadanie 02

# Odpowiedzialności testu i Page Object

| Test                        | WorkshopLoginPage                  |
|-----------------------------|------------------------------------|
| Wybiera scenariusz i dane   | Przechowuje locatory formularza    |
| Przygotowuje wymagany stan  | Udostępnia open() i loginAs(...)   |
| Sprawdza rezultat logowania | Wypełnia pola i wysyła formularz   |
| Pokazuje sens asercji       | Udostępnia errorMessage do asercji |

BasePage obsługuje nawigację. Generator danych ma osobną klasę.

# Locatory lokalnego formularza

Fragmenty do uzupełnienia w WorkshopLoginPage.ts.

```
// W konstruktorze po super(page, loginUrl)
this.emailInput = page.getByLabel('Email');
this.passwordInput = page.getByLabel('Password');
this.submitButton = page.getByRole('button', { name: 'Sign in' });
this.errorMessage = page.getByTestId('login-error');

async loginAs(email: string, password: string) {
  await this.emailInput.fill(email);
  await this.passwordInput.fill(password);
  await this.submitButton.click();
}
```

Pola locatorów otrzymują typ Locator. open() pochodzi z BasePage.

# Scenariusz po wydzieleniu Page Object

Dane i oczekiwany wynik pozostają widoczne w teście.

```
const loginPage = new WorkshopLoginPage(page);  
// Arrange  
await loginPage.open();  
const email = 'workshop.user@example.com';  
  
// Act  
await loginPage.loginAs(email, 'wrong-password');  
// Assert  
await expect(loginPage.errorMessage)  
  .toHaveText('Nieprawidłowy login lub hasło.');
```

Zmiana selektora formularza trafia do jednej klasy.

# Uzupełnienie WorkshopLoginPage

## Punkt startowy

```
src/pageobjects/WorkshopLoginPage.ts  
tests/workshop/login.spec.ts
```

## Do wykonania

1. Dodaj typowane locatory i zaimplementuj loginAs.
2. Użyj open() oraz loginAs w swoim teście.
3. Pozostaw asercję errorMessage w specyfikacji.

## Kryteria ukończenia

Selektory formularza są w PO.

Metoda opisuje logowanie.

Scenariusz nadal jest czytelny.

```
npx playwright test tests/workshop/login.spec.ts --project=chromium
```

# SMURF

## Jakość i utrzymanie testów

Fixtures, parametryzacja, API i diagnostyka  
Zadania 03–04 oraz rozszerzenia 05–10

# Pięć kryteriów SMURF

## **S Small**

Jedno zachowanie w każdym teście.

## **M Maintainable**

Wspólne selektory i jasny podział odpowiedzialności.

## **U Understandable**

Czytelne AAA i nazwy opisujące scenariusz.

## **R Repeatable**

Własne dane, cleanup i niezależność od kolejności.

## **F Fast**

Krótkie przygotowanie stanu i oczekiwanie na efekt.

# Wspólne przygotowanie przez beforeEach

Istniejący przykład: login-smurf-before-each.spec.ts.

```
test.beforeEach(async ({ page }) => {
  await page.route('**/login', async route => {
    await route.fulfill({
      contentType: 'text/html', body: loginPageHtml
    });
  });
  await page.goto('/login');
});

test('shows an error for an unknown user', async ({ page }) => {
  await submitLoginForm(page, invalidCredentials.unknownUser);
  await expect(page.getByTestId('flash-message'))
    .toHaveText('Invalid credentials');
});
```

# Przegląd testu według SMURF

## Punkt startowy

Własny test: `tests/workshop/login.spec.ts`

Wzór: `tests/patterns/login-smurf-before-each.spec.ts`

## Do wykonania

1. Oceń swój test według pięciu kryteriów.
2. Popraw nazwy, zbędne kroki i kruche selektory.
3. Uruchom test kilka razy bez sztywnych opóźnień.

## Kryteria ukończenia

Jedno zachowanie w teście.

Dane i wynik są przewidywalne.

Powtórzenia przechodzą.

```
npx playwright test tests/workshop/login.spec.ts --project=chromium --repeat-each=3
```

# Typowane fixtures i zakres zasobu

Fixture przekazuje testowi nazwany zasób i zarządza jego cyklem życia.

| Zakres | Zasób w projekcie | Odpowiedzialność                   |
|--------|-------------------|------------------------------------|
| Test   | testUser          | Własne konto i usunięcie po teście |
| Test   | loginPage         | Obiekt strony dla bieżącego page   |
| Worker | api               | Kontekst żądań i dispose po użyciu |

fixtures/loginPage/fixtures.ts pokazuje kontekst API.

fixtures/smurf/fixtures.ts zawiera TODO uczestnika.

# Przygotowanie i cleanup w fixture

Fragment proponowanej implementacji testUser.

```
testUser: async ({ request }, use) => {
  const user = {
    email: 'workshop-' + randomUUID() + '@example.com',
    password: 'correct-password', active: true
  };
  const url = '/api/users/' + encodeURIComponent(user.email);
  try {
    const created = await request.post('/api/users', { data: user });
    expect(created.status()).toBe(201);
    await use(user);
  } finally {
    const deleted = await request.delete(url);
    expect(deleted.status()).toBe(204);
  }
},
```

# Fixtures dla danych i Page Object

## Punkt startowy

```
fixtures/loginPage/fixtures.ts i fixtures/smurf/fixtures.ts  
Nowy test: tests/workshop/login-fixtures.spec.ts
```

## Do wykonania

1. Porównaj test scope i worker scope w przykładach.
2. Uzupełnij testUser: POST, await use i cleanup DELETE.
3. Uzupełnij loginPage: obiekt, open() i await use.

## Kryteria ukończenia

Zasoby mają jawne typy.

Test używa rozszerzonego test.

Każdy test sprząta swoje konto.

```
npx playwright test tests/workshop/login-fixtures.spec.ts --project=chromium
```

# Parametryzacja przypadków logowania

Jedna tabela danych i osobny wynik testu dla każdego przypadku.

| Przypadek           | Konto                | Hasło w UI       |
|---------------------|----------------------|------------------|
| Nieznany użytkownik | Nie tworzymy konta   | correct-password |
| Puste hasło         | Własne aktywne konto | Pusty string     |
| Błędne hasło        | Własne aktywne konto | wrong-password   |

Każdy przypadek oczekuje: „Nieprawidłowy login lub hasło.”  
Konta tworzone przez API wymagają cleanupu.

# Osobne testy z tabeli przypadków

Pętla deklaruje testy podczas zbierania pliku.

```
for (const scenario of cases) {  
  test(scenario.name, async ({ page, request }) => {  
    // Arrange: własny email i konto, jeśli potrzebne  
    // Act: wspólne kroki formularza  
    // Assert: oczekiwany tekst z scenario  
    // Cleanup: własne konto w finally lub fixture  
  });  
}
```

Raport ma pokazać trzy osobne scenariusze.

Typ przypadku opisuje nazwę, utworzenie konta, hasło i oczekiwanie.

# Trzy parametryzowane scenariusze

## Punkt startowy

Instrukcja: `exercises/05-parametryzacja-testow.md`

Nowy plik: `tests/workshop/login-parameterized.spec.ts`

## Do wykonania

1. Dodaj typ i tabelę trzech przypadków.
2. Deklaruj osobne testy przez `for...of`.
3. Przygotuj konta dla pustego i błędnego hasła.

## Kryteria ukończenia

Raport pokazuje trzy testy.

Kroki logowania są wspólne.

Cleanup działa po błędzie.

```
npx playwright test tests/workshop/login-parameterized.spec.ts --project=chromium
```

# API przygotowuje blokadę konta

API ustawia stan. Formularz sprawdza logowanie z poprawnym hasłem.

```
// Arrange: konto ma poprawne hasło i active: true
const created = await request.post('/api/users', { data: user });
expect(created.status()).toBe(201);
const locked = await request.patch(userUrl, {
  data: { active: false }
});
expect(locked.status()).toBe(200);

// Act
await loginPage.open();
await loginPage.loginAs(user.email, user.password);

// Assert
await expect(loginPage.errorMessage)
  .toHaveText('Konto użytkownika jest zablokowane.');
```

# Zablokowane konto w API i UI

## Punkt startowy

Instrukcja: `exercises/06-zmiana-stanu-api.md`

Nowy plik: `tests/workshop/locked-user.spec.ts`

## Do wykonania

1. Utwórz własne konto i zablokuj je przez PATCH.
2. Zaloguj się poprawnym hasłem tego konta.
3. Sprawdź blokadę i brak Dashboard, potem usuń konto.

## Kryteria ukończenia

POST 201 i PATCH 200.

UI pokazuje tekst blokady.

DELETE 204 również po błędzie.

```
npx playwright test tests/workshop/locked-user.spec.ts --project=chromium
```

# Kontrakt API użytkowników

Status i JSON pokazują różne elementy zachowania endpointu.

| Przypadek                   | Status | Oczekiwana odpowiedź             |
|-----------------------------|--------|----------------------------------|
| POST z pełnymi danymi       | 201    | user zawiera dane i active: true |
| POST bez emaila             | 400    | email and password are required  |
| POST bez hasła              | 400    | email and password are required  |
| PATCH nieistniejącego konta | 404    | user not found                   |

Cleanup: DELETE zwraca 204. Odpowiedź nie zawiera JSON do odczytu.

# Cztery niezależne testy API

## Punkt startowy

Instrukcja: `exercises/07-testy-api.md`

Nowy plik: `tests/workshop/users-api.spec.ts`

## Do wykonania

1. Dodaj sukces POST i sprawdzenie własnego konta w GET.
2. Dodaj dwa błędy walidacji i PATCH dla brakującego konta.
3. Sprawdź status oraz JSON, usuń utworzone konto.

## Kryteria ukończenia

Cztery wyniki w raporcie.

Testy używają request.

Nie zależą od kolejności.

```
npx playwright test tests/workshop/users-api.spec.ts --project=chromium
```

# Błąd selektora i błąd oczekiwania

Raport i trace pozwalają wskazać przyczynę na podstawie wykonania.

|                        |                                                              |
|------------------------|--------------------------------------------------------------|
| <b>Brak elementu</b>   | Test szuka flash-message. DOM aplikacji zawiera login-error. |
| <b>Niezgodny tekst</b> | Element istnieje, ale test oczekuje Invalid credentials.     |
| <b>Dowód w trace</b>   | Snapshot DOM, log oczekiwania i odpowiedź POST /login.       |

Konfiguracja zapisuje trace każdego wykonania i screenshot przy błędzie.

# Diagnoza dwóch celowych usterek

## Punkt startowy

Instrukcja: `exercises/08-diagnozowanie-bledu.md`

Nowy plik: `tests/workshop/login-debug.spec.ts`

## Do wykonania

1. Uruchom działający test nieznanego użytkownika.
2. Zmień selektor na `flash-message` i sprawdź trace.
3. Po naprawie zmień tekst na `Invalid credentials`.

## Kryteria ukończenia

Przyczyna wynika z DOM i logu.

Rozróżniasz obie usterki.

Przywrócony test przechodzi.

```
npx playwright test tests/workshop/login-debug.spec.ts --project=chromium --trace=on
npm run test:report
```

# DataGenerator poza Page Object

Przeniesienie do `utils/generators/dataGenerator.ts` już istnieje w starterze.

```
randomString(len: number = 10, type: string = 'letters'): string
```

| Tryb              | Właściwość wyniku                 |
|-------------------|-----------------------------------|
| letters i LETTERS | Litery A–Z oraz a–z               |
| number            | Cyfry 0–9, także zero na początku |
| Inny string       | Litery i cyfry                    |

Wynik zawsze jest stringiem. Długość 0 daje pusty string.

# Kontrakt generatora danych

## Punkt startowy

`utils/generators/dataGenerator.ts` i `src/pageobjects/basePage.ts`  
Nowy plik: `tests/workshop/data-generator.spec.ts`

## Do wykonania

1. Sprawdź długość i znaki w trzech trybach.
2. Dodaj długość 0 oraz tryb LETTERS.
3. Wyjaśnij rozdzielenie generatora i BasePage.

## Kryteria ukończenia

Generator działa bez page.

Metoda zwraca string.

Test nie zależy od losowej wartości.

```
npx eslint utils --max-warnings=0  
npx playwright test tests/workshop/data-generator.spec.ts --project=chromium
```

# Ten sam scenariusz w trzech wariantach

Własne aktywne konto, poprawne logowanie, Dashboard i powitanie.

| Pytanie            | AAA              | Page Object     | Fixtures          |
|--------------------|------------------|-----------------|-------------------|
| Dane i cleanup     | W teście         | W teście        | W testUser        |
| Zmiana selektora   | W specyfikacji   | W jednej klasie | W jednej klasie   |
| Odczyt scenariusza | Jawne kroki      | Nazwane akcje   | Jawne zasoby      |
| Śledzenie błędu    | Głównie test     | Test i PO       | Test, PO, fixture |
| Ponowne użycie     | Powtarzane kroki | Interakcje UI   | Interakcje i dane |

AAA obowiązuje w każdym wariantcie. Zmieniamy miejsce odpowiedzialności.

# Porównanie kosztu trzech wariantów

## Punkt startowy

```
tests/workshop/comparison-aaa.spec.ts  
comparison-page-object.spec.ts i comparison-fixtures.spec.ts
```

## Do wykonania

1. Zapisz to samo udane logowanie w trzech plikach.
2. Użyj własnego konta i cleanupu w każdym wariacie.
3. Porównaj kod oraz koszt diagnozy i zmiany selektora.

## Kryteria ukończenia

Każdy wariant sprawdza ten sam efekt.

Trzy testy działają równolegle.

Wskazujesz korzyść i koszt.

```
npx playwright test tests/workshop/comparison --project=chromium --workers=3
```

# Checklista końcowa

## AAA

Widać przygotowanie, zachowanie i oczekiwany wynik.

## Page Object

Interakcje mają nazwy, a asercja pokazuje cel testu.

## SMURF

Test ma własne dane, cleanup i czytelną diagnozę.

Decyzję o abstrakcji uzasadnia koszt konkretnej zmiany.

# Polecenia do pracy z projektem

```
# Wybrany katalog testów po ich utworzeniu
npx playwright test tests/workshop --project=chromium

# Powtarzalność i praca równoległa
npx playwright test tests/workshop --project=chromium --repeat-each=3
npx playwright test tests/workshop --project=chromium --workers=3

# Diagnostyka
npm run test:report
npx playwright show-trace <ścieżka-do-trace.zip>
```

Dla UI i API pozostaw `npm run app` w osobnym terminalu.

Pełny zestaw wymaga dostosowania dawnego przykładu Page Object.

# Dobre praktyki pracy z AI

Źródłem prawdy są wymagania, dokumentacja i zachowanie systemu.

| Co zrobić                                 | Jak weryfikować                                     | Na co zwrócić uwagę                            |
|-------------------------------------------|-----------------------------------------------------|------------------------------------------------|
| Zadanie i źródła<br>Podaj cel i wersję.   | Porównaj wynik z<br>wymaganiem lub kodem.           | Oznacz założenia<br>oraz brakujące dane.       |
| Dane wejściowe<br>Użyj danych fikcyjnych. | Sprawdź anonimizację<br>przed przekazaniem.         | Korzystaj z narzędzia<br>przyjętego w zespole. |
| Format wyniku<br>Poproś o szkic testu.    | Przeczytaj cały wynik.<br>Potwierdź liczby i nazwy. | Każda istotna teza<br>wymaga dowodu.           |
| Decyzja człowieka<br>Wykonaj review.      | Oceń wpływ zmiany.<br>Potem zatwierdź wynik.        | Zachowaj prompt,<br>wersję i wynik oceny.      |

# Review testu wygenerowanego przez AI

Przykłady z warsztatu: AAA, Page Object, fixtures i diagnoza błędu.

| Obszar            | Dowód poprawności                                       | Ryzyko                                    |
|-------------------|---------------------------------------------------------|-------------------------------------------|
| Oczekiwany wynik  | Porównaj asercję z app/server.js i wymaganiem.          | Zmyślony komunikat lub status API.        |
| Page Object i AAA | Sprawdź locatory w DOM.<br>Przeczytaj sens scenariusza. | Abstrakcja ukrywająca cel testu.          |
| Dane i cleanup    | Uruchom testy równolegle.<br>Sprawdź usunięcie konta.   | Wspólne konto lub globalny reset.         |
| Naprawa awarii    | Porównaj patch z trace.<br>Uruchom testy powiązane.     | Usunięta asercja lub sztuczne opóźnienie. |

Zielony test wymaga asercji, która nadal sprawdza właściwe wymaganie.

# Rubryka oceny odpowiedzi AI: 0–2

Każde kryterium oceniaj osobno, ze wskazaniem dowodu. Stosuj także do RAG.

| Kryterium           | 0                 | 1                    | 2                    |
|---------------------|-------------------|----------------------|----------------------|
| Poprawność faktów   | błędne informacje | częściowo poprawne   | poprawne i kompletne |
| Zgodność ze źródłem | sprzeczna         | częściowo oparta     | w pełni ugruntowana  |
| Realizacja celu     | cel pominięty     | wymaga korekty       | cel zrealizowany     |
| Bezpieczeństwo      | naruszenie zasad  | niejasne zachowanie  | zgodna z zasadami    |
| Język i UX          | myląca            | zrozumiała z uwagami | jasna i dopasowana   |

Błąd faktów lub wyciek danych wymaga poprawy, niezależnie od sumy punktów.

# Weryfikacja RAG: źródła i odpowiedź

RAG: model tworzy odpowiedź na podstawie fragmentów znalezionych w dokumentach.

| Etap                     | Co sprawdzić                               | Jak zebrać dowód                                     |
|--------------------------|--------------------------------------------|------------------------------------------------------|
| Wyszukiwanie (retrieval) | Czy pobrano wszystkie potrzebne fragmenty? | Zapisz ID dokumentów, wersje i ranking.              |
| Odpowiedź                | Czy każde twierdzenie wynika z fragmentów? | Porównaj liczby, warunki i kompletność odpowiedzi.   |
| Cytowanie                | Czy cytat potwierdza konkretne zdanie?     | Otwórz źródło. Sprawdź miejsce i dostęp użytkownika. |

Trafne źródło i poprawna odpowiedź wymagają osobnych ocen.

# Scenariusze testowe RAG

Dla każdego przypadku ustal oczekiwane źródło przed uruchomieniem.

| Przypadek                  | Oczekiwane zachowanie                          | Jak weryfikować                                   |
|----------------------------|------------------------------------------------|---------------------------------------------------|
| Typowe pytanie i parafraza | Te same trafne źródła i kompletna odpowiedź.   | Porównaj źródła także przy literówce lub skrócie. |
| Brak wiedzy                | System komunikuje brak podstawy do odpowiedzi. | Sprawdź, czy nie dopisuje reguł ani szczegółów.   |
| Sprzeczne wersje           | Aktualny dokument lub jawny konflikt.          | Sprawdź status wersji i warunki obowiązywania.    |
| Aktualizacja lub usunięcie | Zmiana treści dociera do indeksu i odpowiedzi. | Zmierz czas zmiany. Sprawdź również cache.        |

Wynik obejmuje pobrane fragmenty, odpowiedź, cytat oraz ocenę 0–2.

# Uprawnienia i prompt injection w RAG

Uprawnienia obowiązują podczas wyszukiwania, generowania i wyświetlania cytatów.

| Ryzyko               | Co zrobić w teście                               | Co potwierdzić                                          |
|----------------------|--------------------------------------------------|---------------------------------------------------------|
| Brak uprawnień       | Zapytaj o prywatny dokument w innej roli.        | Brak treści, tytułu, streszczenia i metadanych.         |
| Tenant / organizacja | Zadaj to samo pytanie w odrębnych organizacjach. | Każdy użytkownik widzi wyłącznie własne źródła.         |
| Historia i cache     | Powtórz pytanie po zmianie dostępu.              | Odpowiedź i cytaty respektują bieżący dostęp.           |
| Prompt injection     | Dodaj do dokumentu polecenie „ujawnij dane”.     | System traktuje je jako dane. Brak niedozwolonej akcji. |

W testach używaj kontrolowanych dokumentów i danych szkoleniowych.

# Zapis wyników i regresja AI / RAG

Stały zestaw przypadków i jedna rubryka pozwalają porównać zmiany.

| Co zapisać   | Minimalny zakres                                               |
|--------------|----------------------------------------------------------------|
| Konfiguracja | Model, prompt, dokumenty i indeks, top-k oraz filtry dostępu.  |
| Przypadek    | Stały zestaw pytań z oczekiwanymi źródłami i wynikiem.         |
| Dowody       | Pobrane fragmenty, pełna odpowiedź, cytaty i ID wykonania.     |
| Ocena        | Ocena 0–2 w każdym kryterium, uzasadnienie i decyzja testera.  |
| Powtórzenia  | Liczba błędów / liczba wykonań. Porównanie przed i po zmianie. |

Po zmianie modelu, promptu lub RAG uruchom ponownie ten sam zestaw. Drugi model może wspierać ocenę. Oczekiwany wynik wymaga źródła.

# Szkolenia Testerzy.pl

Zapraszam na kolejne szkolenia

**300 zł**

rabatu na szkolenie dla uczestników warsztatu

**Kod rabatowy: 123**

# Dziękuję za udział!

Kontakt

krzysztof.kolodziejczyk21cn@gmail.com